

Copilot V1 — Itération 2

Architecture Technique

Version : 1.0

Statut : Document de référence V1

1. Objectif

L'architecture technique de Copilot doit répondre à quatre objectifs.

- Construire rapidement un produit robuste.
- Pouvoir accueillir facilement plus de 1 000 utilisateurs.
- Être suffisamment modulaire pour faire évoluer le moteur.
- Éviter toute réécriture majeure lors des prochaines versions.

Le MVP ne doit jamais être une architecture temporaire.

Chaque composant doit pouvoir être conservé en production.

2. Stack technique

Frontend

- Flutter
- Dart

Pourquoi Flutter ?

- Une seule base de code
 - Android
 - iOS
 - Excellentes performances
 - Grande communauté
 - Déploiement rapide
-

Backend

Supabase

Le backend fournit :

- Authentification
- API
- PostgreSQL
- Stockage
- Edge Functions
- Sécurité
- Temps réel (si nécessaire plus tard)

Aucun serveur Node dédié n'est nécessaire pour la V1.

Base de données

PostgreSQL

Pourquoi PostgreSQL ?

- Base relationnelle extrêmement robuste
- Requêtes complexes
- Indexation performante
- Versionnement facile
- Compatible avec l'évolution future du moteur

Toutes les données métier seront stockées dans PostgreSQL.

Langage serveur

TypeScript

Utilisé uniquement pour :

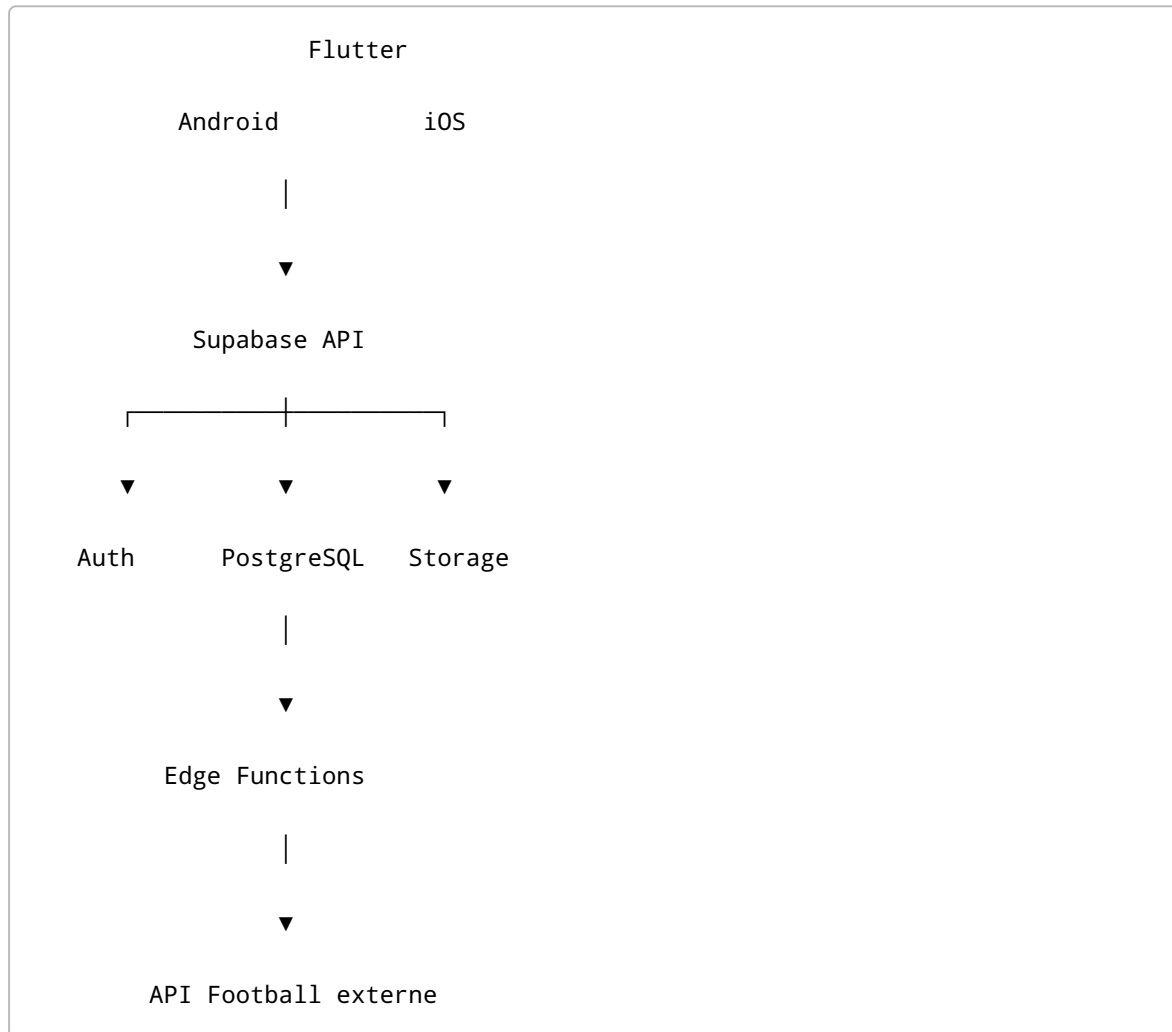
- Edge Functions
 - traitements planifiés
 - appels API externes
 - calculs serveur
-

Hébergement

Supabase Cloud

L'objectif est de ne gérer aucune infrastructure serveur.

3. Architecture générale



4. Répartition des responsabilités

Flutter

Flutter est responsable de :

- l'interface utilisateur
- la navigation
- l'affichage des rencontres
- les formulaires
- les appels API
- la gestion de session
- le cache local

Flutter ne contient jamais le moteur de décision.

PostgreSQL

PostgreSQL est responsable de :

- toutes les données métier
- les utilisateurs
- les rencontres
- les statistiques
- les préférences
- les tickets
- le moteur
- les règles
- les historiques

Toutes les informations importantes doivent être persistées.

Edge Functions

Les Edge Functions exécutent :

- récupération des données football
- traitements planifiés
- calcul du moteur
- synchronisation
- génération des recommandations

Elles représentent la couche métier du backend.

5. Architecture Flutter

Le projet Flutter est organisé par fonctionnalités.

```
lib/  
  
app/  
  
core/  
  
shared/  
  
features/
```

app/

Contient :

- configuration générale
 - navigation
 - thème
 - injection de dépendances
-

core/

Contient :

- erreurs
 - constantes
 - configuration
 - outils communs
-

shared/

Contient :

- widgets réutilisables
 - composants UI
 - design system
 - modèles communs
-

features/

Chaque fonctionnalité possède son propre module.

```
features/  
  
authentication/  
  
onboarding/  
  
matches/  
  
match_details/  
  
ticket_builder/  
  
ticket_history/
```

```
profile/
```

Chaque module contient :

```
presentation/
```

```
domain/
```

```
data/
```

Cette architecture permet de faire évoluer chaque fonctionnalité indépendamment.

6. Authentification

La V1 supporte :

- création de compte
- connexion
- déconnexion
- récupération de mot de passe
- maintien de session

Le profil utilisateur est entièrement séparé du système d'authentification.

7. Sécurité

Chaque utilisateur ne peut accéder qu'à ses propres données.

Les règles de sécurité sont appliquées directement dans PostgreSQL via Row Level Security.

Les tables publiques (équipes, compétitions, rencontres...) sont uniquement consultables.

Les tables privées (tickets, préférences, onboarding...) sont filtrées par utilisateur.

8. Stockage

Deux types de stockage existent.

Données relationnelles

Toutes les données importantes sont stockées dans PostgreSQL.

Exemples :

- préférences
 - tickets
 - sélections
 - moteur
 - statistiques
-

Fichiers

Supabase Storage est réservé aux fichiers.

Exemples :

- avatars
- captures
- exports PDF

Les fichiers ne contiennent jamais de logique métier.

9. Cache local

L'application possède un cache local.

Il sert uniquement à améliorer l'expérience utilisateur.

Le cache peut contenir :

- session
- thème
- dernières rencontres consultées
- ticket en cours
- filtres

Le cache n'est jamais considéré comme la source officielle des données.

PostgreSQL reste la référence.

10. Architecture du moteur

Le moteur est entièrement séparé de Flutter.

Flutter

↓

API

↓

Moteur

↓

Résultats

↓

Flutter

Flutter ne calcule jamais :

- les recommandations
- les scores
- les priorités
- les compatibilités

Flutter affiche uniquement les résultats.

11. Flux quotidien

Chaque jour :

API Football

↓

Import des rencontres

↓

Import des statistiques

↓

Calcul des métriques

↓

Calcul des signaux

↓

Stockage

↓

Classement personnalisé

↓

Application mobile

Le moteur travaille avant que l'utilisateur ouvre l'application.

Ainsi, l'expérience reste fluide.

12. Performances

Le moteur ne doit jamais recalculer toutes les statistiques pour chaque utilisateur.

Le calcul est découpé en deux étapes.

Étape 1

Calcul universel.

Une seule fois.

Exemple :

Arsenal

Forme domicile

92

Everton

Forme extérieure

34

Ces valeurs sont calculées une seule fois.

Étape 2

Calcul personnalisé.

Le moteur applique les préférences utilisateur.

Exemple :

Utilisateur A

Importance domicile

100 %

Utilisateur B

Importance domicile

40 %

Le moteur réutilise les mêmes statistiques.

Il ne les recalcule jamais.

Cette séparation garantit des performances élevées.

13. Versionnement

Trois éléments doivent être versionnés.

Le moteur

Chaque évolution du moteur possède son numéro de version.

Exemple :

1.0

1.1

1.2

Les règles

Chaque règle appartient à une version du moteur.

L'onboarding

Chaque questionnaire possède également une version.

Ainsi, un utilisateur peut conserver son ancien profil même après une mise à jour.

14. Journalisation

Toutes les opérations importantes sont enregistrées.

Exemples :

- connexion
- modification du profil
- création d'un ticket
- suppression
- changement de marché
- recalcul du moteur

Cette journalisation facilitera le diagnostic et l'évolution du produit.

15. API Football

Le fournisseur de données doit être totalement indépendant.

Le moteur ne dépend jamais d'une API spécifique.

Toutes les données importées sont normalisées dans PostgreSQL.

Ainsi, changer de fournisseur ne nécessite pas de modifier Flutter ni le moteur.

16. Déploiement

Développement

Flutter

↓

Supabase Development

Préproduction

Flutter

↓

Supabase Staging

Production

Flutter

↓

Supabase Production

Trois environnements totalement séparés.

17. Tests

Trois niveaux de tests sont prévus.

Flutter

- widgets
 - navigation
 - affichage
-

Backend

- Edge Functions
 - import des données
 - traitements
-

Moteur

Chaque règle devra posséder ses propres tests.

Une évolution du moteur ne devra jamais modifier involontairement le comportement d'une règle existante.

18. Scalabilité

L'architecture est pensée dès la V1 pour évoluer.

Les principaux facteurs de montée en charge sont :

- augmentation du nombre de compétitions ;
- augmentation du nombre de rencontres ;
- augmentation du nombre d'utilisateurs ;
- enrichissement du moteur.

Aucun changement d'architecture ne doit être nécessaire pour atteindre plusieurs milliers d'utilisateurs.

L'objectif de cette V1 est de disposer de fondations suffisamment solides pour permettre l'évolution du produit pendant plusieurs années sans remise en cause des choix techniques initiaux.

19. Décisions d'architecture

Les décisions suivantes sont considérées comme définitives pour la V1.

- ✓ Flutter comme framework mobile unique.
- ✓ Supabase comme plateforme backend.
- ✓ PostgreSQL comme source unique de vérité.
- ✓ Architecture modulaire par fonctionnalités.
- ✓ Moteur de décision entièrement séparé de l'interface.
- ✓ Toutes les données métier normalisées dans PostgreSQL.
- ✓ Versionnement du moteur et de l'onboarding.
- ✓ Aucun calcul métier important réalisé côté mobile.

Ces principes serviront de référence pour l'ensemble du développement de Copilot.